EngageNet: Multi-Modal Engagement Regression with Ordered State-Space Fusion

Lado Turmanidze

August 23, 2026

Abstract

EngageNet is a multimodal state-space architecture for real-time engagement regression in dyadic interactions. It combines per-modality bidirectional selective state-space models (BiMamba) with a differentiable Gumbel-Sinkhorn permutation layer for inter-modal fusion, and outputs calibrated predictions via Beta regression heads. A test-time adaptation mechanism leveraging predictive variance as an entropy proxy enables robust deployment under distribution shift across corpora (NoXi, NoXi+J, MPIIGroupInteraction). The system is evaluated using the Concordance Correlation Coefficient (CCC) as the primary metric and the Conditional Demographic Disparity (CDD) as a fairness constraint.

1 Architecture

1.1 Input

Let $x = \{x_1, \ldots, x_M\}$ with $x_i \in \mathbb{R}^{C_i \times L}$ for all $i \in \{1, \ldots, M\}$, where C_i is the number of acquisition channels for the i-th modality and L represents the number of time steps of the input signal.

1.1.1 Dyadic Structure and Role Streams

Each session in the NoXi [17] family of corpora - including the NoXi+J extension [18] - is a screen-mediated dyadic interaction between an *expert* and a *novice*, recorded independently (separate cameras, separate microphones). Every feature type therefore arrives as two parallel streams, one per role, and both carry their own frame-wise engagement annotation.

We denote by $\mathcal{F}$ the set of active feature types (e.g. eGeMAPS v2, W2v-BERT 2.0, OpenFace2, OpenPose) and by $\mathcal{R} = \{\text{expert}, \text{novice}\}$ the set of roles. The index i in x_i ranges over the Cartesian product $\mathcal{F} \times \mathcal{R}$, so that

$$M = |\mathcal{F}| \times |\mathcal{R}| = 2|\mathcal{F}|.$$

With the four core feature types this yields $M = 8$ input streams. Every subsequent occurrence of M in this document - including the modality-ordering permutation of Section 3 - refers to these role-feature pairs, not to feature types alone.

Weight sharing across roles. The two roles observe the same physical signal semantics (an eGeMAPS vector means the same thing whether it was extracted from the expert's or the novice's microphone). Accordingly, the InitEncoder, the uniform channel projection, the intra-modal BiMamba block, and the unimodal Beta head are instantiated *once per feature type* and applied to both roles. Only the inter-modal stage distinguishes the roles, since the permutation is learned over all M role-feature pairs jointly. This halves the parameter count relative to per-role encoders and forces the shared representation to be role-agnostic.

Target construction. Write y_t^e and y_t^n for the engagement annotations of the expert and the novice at frame t, each in $[0, 1]$. The challenge asks for both separately [13], and we predict both. Suppose both annotations have the same variance σ^2 and correlation r with each other. Scaling a random variable by $\frac{1}{2}$ scales its variance by $\frac{1}{4}$, and the variance of a sum is the sum of the variances plus twice the covariance, so

$$\mathrm{Var}(\bar{y}) = \tfrac{1}{4}\left[\mathrm{Var}(y^e) + \mathrm{Var}(y^n) + 2\,\mathrm{Cov}(y^e, y^n)\right] = \tfrac{1}{4}\left[\sigma^2 + \sigma^2 + 2r\sigma^2\right] = \frac{\sigma^2(1+r)}{2}$$

using $\mathrm{Cov}(y^e, y^n) = r\sigma^2$. The corpus paper [18] measures exactly this correlation and finds it to be small and, in three of four language sets, negative: $r = -0.19$ for German, -0.07 for English, -0.05 for French, and $+0.51$ for Japanese, the last attributed to a stronger tendency toward interpersonal harmony. Substituting these values, the averaged target retains only 41%, 47% and 48% of the variance in the European sets respectively, and 76% in the Japanese set - so on most of the corpus more than half the signal is destroyed before training begins. Since CCC is normalised by the variance of the target (Section 4.1), what is flattened away here cannot be recovered later.

Predicting both roles needs no new machinery, because the two participants are never mixed on the way in: their recordings enter as separate streams and stay separately indexed throughout. The network therefore ends with one output head per role rather than one shared head, each producing that participant's engagement curve, and each auxiliary per-modality head is trained against the participant whose stream it reads.

1.1.2 Input Standardisation

The raw feature streams occupy vastly different numerical ranges: eGeMAPS v2 $\in [-2.6 \times 10^5, 2.6 \times 10^5$ and OpenFace2 $\in [-3 \times 10^4, 3 \times 10^4]$, while W2v-BERT 2.0 embeddings lie within $[-4, 4]$. These magnitudes compound through the encoder and state-space stages, reaching $\sim 10^{11}$ at the cross-modal block and driving the Beta shape parameters to $\sim 10^{14}$, at which point the negative log-likelihood is numerically meaningless and optimisation fails.

We therefore standardise every channel to zero mean and unit variance before it enters the network:

$$\tilde{x}_i^{(c)} = \frac{x_i^{(c)} - \mu_i^{(c)}}{\sigma_i^{(c)}}, \qquad c \in \{1, \ldots, C_i\},$$

where the per-channel statistics are accumulated in double precision over the **training split only** and reused unchanged for validation and test. Channels whose standard deviation falls below 10^{-8} (constant channels) are centred but not rescaled, avoiding division by zero. Because the two roles share encoder weights, they necessarily share normalisation statistics as well. The statistics are computed once per corpus.

1.2 Output

For each frame t of the encoded sequence the network emits two strictly positive shape parameters (α_t, β_t) that define a Beta predictive density over the bounded engagement scale,

$$p(y_t \mid x) = \mathrm{Beta}\big(\alpha_t(x),\, \beta_t(x)\big), \qquad y_t \in [0, 1].$$

The scalar point prediction is the predictive mean:

$$\hat{y}_t = \hat{\mu}_t = \frac{\alpha_t}{\alpha_t + \beta_t} \in (0, 1),$$

while the predictive variance supplies the uncertainty signal consumed by the test-time adaptation filter (Section 2.3). In contrast to [1], where y is a discrete class label represented as a

one-hot vector $p(y|x) \in \mathbb{R}^N$ over N categories, engagement estimation is a *continuous bounded regression* problem: there is no category axis, and the network output for a window of length L' has shape $(L', 2)$ rather than (L', N).

In addition to the fused multimodal head, the network exposes one *unimodal* Beta head per feature type, attached to the corresponding intra-modal representation u_i before fusion. These heads produce (α_i, β_i) and serve two purposes: deep supervision during training (Section 2.5) and the teacher-student consistency objective during test-time adaptation (Section 2.6).

One such head is instantiated per role, so a single forward pass yields $\left(\alpha_t^r, \beta_t^r\right)$ for each $r \in \mathcal{R}$. The two heads read the same fused representation and differ only in their read-out weights.

1.3 Inner Workings

We need to capture contextual information from physiological time series simultaneously, and a unidirectional approach cannot fully capture the complex temporal patterns. The multimodal BiMamba network consists of two modules: the intra-modal BiMamba module and the inter-modal BiMamba module. The former is designed to model each modality independently, while the latter effectively fuses multimodal features.

1.3.1 Intra-modal BiMamba Module

First, an initial encoder $h_i = \text{InitEncoder}_i(\tilde{x}_i) \in \mathbb{R}^{L' \times C'}$ for all $i \in \{1, \ldots, M\}$ is used to extract shallow features from a specific modality, where C' is the number of output channels and L' is the feature length of the output signal.

The transposition of dimensions from $C_i \times L$ to $L' \times C'$ physically aligns the data structure with the sequence-first requirement of autoregressive models. By enforcing the sequence length L' as the primary operational dimension, the subsequent state-space and 1D convolutional operators can process the temporal dimension across the channel feature space.

InitEncoder internals. Each InitEncoder is a shallow three-stage block applied to the standardised input:

$$h_i = \text{InitEncoder}_i(\tilde{x}_i) = f_i\Big(\sigma\big(\text{BN}\big(\text{Conv1D}_i(\tilde{x}_i^\top)\big)\big)\Big),$$

where Conv1D_i is a temporal convolution with kernel size k_i, stride s_i and `SAME` padding mapping $C_i \to C_i'$ channels (the stride controls the downsampling $L \to L'$); BN is batch normalisation [16] over the channel axis, whose running statistics are frozen at evaluation time and which later serves as a surgical adaptation target (Section 2.6); $\sigma(x) = \frac{x}{1+e^{-x}}$ is the SiLU activation function, and f_i is the uniform channel projection defined in Section 3.2.1, mapping $C_i' \to C'$ so that all modalities share a common width. Encoder weights are shared between the expert and novice streams of the same feature type.

The BiMamba module operates on h_i as follows. Let $\text{LN}(\cdot)$ denote layer normalisation [15].

1. **Pre-normalisation.** The block input is normalised before any projection:

$$\bar{h}_i = \text{LN}(h_i) \in \mathbb{R}^{L' \times C'}.$$

   Without this step the residual stream is unnormalised at every depth and activation magnitudes compound multiplicatively through the stacked intra-modal and cross-modal blocks. Empirically the absolute activation maximum grew from ~ 10 at the frontend to $\sim 6 \times 10^{11}$ at the cross-modal stage, saturating the Beta heads and destroying the gradient signal. Pre-normalisation keeps the block input at unit scale while leaving the residual path (step 4) untouched.

2. The output of the gating mechanism is $g_i = \sigma(W_i^g \bar{h}_i + b_i^g)$, where W_i^g is the weight matrix, b_i^g is the bias, and $\sigma(x) = \dfrac{x}{1+e^{-x}}$. Intuitively, g_i highlights engagement-relevant information while suppressing noise and redundancy.

3. BiMamba models temporal dependencies from both forward and backward perspectives using state-space modeling, thereby capturing richer contextual dynamics:

$$h_i^{\rightarrow} = g_i \otimes \mathrm{SSM}_{\rightarrow}(\sigma(\mathrm{DWConv1D}_{\rightarrow}(W_i^h \bar{h}_i + b_i^h))) \in \mathbb{R}^{L' \times C'}$$

$$h_i^{\leftarrow} = g_i \otimes \mathrm{Rev}_t\Big(\mathrm{SSM}_{\leftarrow}(\sigma(\mathrm{DWConv1D}_{\leftarrow}(\mathrm{Rev}_t(W_i^h \bar{h}_i + b_i^h))))\Big) \in \mathbb{R}^{L' \times C'}$$

where $h_i^{\rightarrow}$ and $h_i^{\leftarrow}$ are the feature representations of the i-th modality in forward and backward modeling. $\mathrm{Rev}_t(\cdot)$ denotes flipping along the time dimension L'.

Note that the time reversal is applied to the backward SSM output *before* gating, so that the gate g_i - which is computed once, in forward time order - modulates the feature belonging to the same time step t in both branches. Applying Rev_t after gating would instead pair the gate at step t with the feature at step $L' - t$, misaligning the two.

The operator $\otimes$ denotes the Hadamard product (element-wise multiplication), which explains why the dimensions remain strictly unchanged, unlike a tensor product. W_i^h represents the weight matrix and b_i^h represents the bias. DWConv1D denotes a *depthwise* temporal convolution of kernel size D_C (one filter per channel, i.e. `feature_group_count` equal to the channel width), following [2]; this keeps the convolution cost linear in the channel dimension.

$\mathrm{SSM}(\cdot)$ represents the Selective State Space Model. Formally, it maps a 1-dimensional sequence through a continuous linear time-invariant (LTI) system defined by the state equation $\xi'(t) = A\xi(t) + Bu(t)$ and output $y(t) = C\xi(t)$. To operate on discrete sequence steps, the continuous transition operator A and input operator B are discretized via a timescale parameter Δ. The exact discretisation yields $\bar{A} = \exp(\Delta A)$ and $\bar{B} = (\Delta A)^{-1}(\bar{A} - I)\Delta B$. Following the reference Mamba implementation [2], we adopt the first-order (Euler) simplification of the input operator,

$$\bar{A} = \exp(\Delta A), \qquad \bar{B} \approx \Delta B,$$

which is accurate to $O(\Delta^2)$ and avoids the matrix inverse $(\Delta A)^{-1}$.

Diagonal state transition. Following [2], the state-space model is applied independently to each of the C' channels, and the state matrix of every channel is restricted to be *diagonal*. Channel d thus carries N state coordinates evolving under $A^{(d)} = \mathrm{diag}(a_{d,1}, \ldots, a_{d,N})$, and because a diagonal matrix is fully determined by its diagonal, no $N \times N$ array is ever materialised: the entire parameter collapses to the $C' \times N$ array $A = (a_{d,n})$, one row of N entries per channel. Each entry governs how quickly one state coordinate forgets, which is why we refer to these entries as decay rates. The update is then elementwise rather than a matrix product, costing $O(N)$ per channel per step instead of $O(N^2)$ and storing $C'N$ parameters instead of $C'N^2$. Diagonality is not merely an efficiency measure: it is what allows the recurrence to be evaluated by a parallel scan, as shown below.

Parameterisation of A. The state transition matrix is not learned directly. Instead we learn $\log(-A)$ and recover

$$A = -\exp\big(A_{\log}\big) \in \mathbb{R}^{C' \times N}, \qquad A_{\log} \text{ initialised to } \mathbf{0},$$

which constrains $A < 0$ (elementwise). This guarantees $\bar{A} = \exp(\Delta A) \in (0, 1)$, so the recurrence is contractive and the hidden state cannot diverge - a stability property that

an unconstrained A does not provide. Separate $A_{\log}$ parameters are maintained for the forward and backward directions.

Temporal receptive field. The decay rate and the timestep jointly determine how current input can influence future output. Consider the following recurrence:

$$\xi_t = \bar{A}\,\xi_{t-1} + b_t, \qquad b_t := \bar{B}_t u_t,$$

which is *first-order*: the state at step t is determined solely by the previous state ξ_{t-1} and the current input alone, with no explicit dependence on ξ_{t-2} or earlier. Suppose the input does not exist after step t_0 ($u_t = 0 \implies b_t = 0$) for $t > t_0$. Unrolling from ξ_{t_0},

$$\xi_{t_0+1} = \bar{A}\,\xi_{t_0}, \qquad \xi_{t_0+2} = \bar{A}^2\xi_{t_0}, \qquad \ldots, \qquad \xi_{t_0+k} = \bar{A}^{\,k}\,\xi_{t_0}.$$

Since $\bar{A} = \exp(\Delta A) \in (0, 1)$, the retained state shrinks geometrically; $\bar{A}$ is called the *decay* factor. Equivalently, and this is the reading we need, the influence of an input entering at step t_0 on the state k steps later carries the weight $\bar{A}^{\,k}$: this is exactly the accumulated multiplier that appears in the associative scan derivation below. The *memory* of the block is the profile of these weights, and its *effective memory* τ_{eff} is the conventional single-number summary - the number of steps after which the weight has fallen to e^{-1} of its initial value. This e^{-1} convention is the standard definition of the time constant of an exponentially decaying response in linear systems theory [20], and we adopt it unchanged. Solving

$$\bar{A}^{\,\tau_{\text{eff}}} = e^{-1} \implies \tau_{\text{eff}} \ln \bar{A} = -1 \implies \tau_{\text{eff}} = \frac{-1}{\ln \bar{A}},$$

and substituting $\ln \bar{A} = \Delta A = -\Delta|A|$ (recall $A < 0$ by construction),

$$\boxed{\tau_{\text{eff}} = \frac{1}{\Delta\,|A|}}$$

measured in units of encoder output steps.

Why the initialisation matters. Initialising $A_{\log} = \mathbf{0}$ and $s_\Delta = \mathbf{0}$ gives $|A| = \exp(0) = 1$ and, since the input-dependent projection contributes negligibly at initialisation, $\Delta \approx \text{softplus}(0) = \ln 2 \approx 0.693$. Hence

$$\tau_{\text{eff}} = \frac{1}{0.693 \times 1} \approx 1.44 \text{ steps}$$

The output sequence retains the input rate of 25 Hz and one step spans $1/25$ s $= 40$ ms; the receptive field is therefore

$$1.44 \times 40\,\text{ms} \approx 58\,\text{ms}$$

A uniform $A_{\log}$ compounds the difficulty: every one of the N state coordinates then carries the same decay rate, so the state dimension supplies N copies of a single timescale rather than a spread of them, entirely defeating the purpose of $N > 1$.

S4D-Real initialisation. We therefore adopt the real-valued diagonal initialisation of [19], used in the reference Mamba implementation [2]: setting

$$A_{\log}[d, n] = \ln(n + 1) \quad \rightarrow \quad a_{d,n} = -(n + 1), \qquad n = 0, \ldots, N - 1,$$

gives each channel a geometric ladder of decay rates rather than a single one. The timestep bias is initialised so that Δ is log-uniform on $[\Delta_{\min}, \Delta_{\max}] = [10^{-3}, 10^{-1}]$, obtained by inverting the softplus,

$$s_\Delta = \ln\left(e^{\Delta} - 1\right),$$

which is the exact inverse since $\text{softplus}(s_\Delta) = \ln(1 + e^{s_\Delta}) = \Delta$. The resulting receptive fields $\tau_{\text{eff}} = \frac{1}{\Delta|A|}$ span from a fraction of a step to of order 10^3 steps - tens of seconds at 25 Hz - allowing the selection mechanism to choose an integration horizon per channel and per input rather than beginning pinned to a single very short one.

Unlike traditional LTI systems, Mamba introduces *selection* by making the discretized parameters $(\bar{B}, C, \Delta)$ functions of the input u_t. Specifically:

$$\Delta_t = \text{softplus}(s_\Delta + \text{Linear}(u_t))$$

$$B_t = \text{Linear}(u_t), \quad C_t = \text{Linear}(u_t)$$

where $\text{softplus}(x) = \ln(1 + e^x)$ and $s_\Delta \in \mathbb{R}^{C'}$ is a learned, input-independent bias term that defines the baseline timescale of the system. The inclusion of s_Δ within the *softplus* function ensures that the resulting Δ_t remains strictly positive, which is required for the stability of the discretization. This selection mechanism allows the discrete transition $\xi_t = \bar{A}_t \xi_{t-1} + \bar{B}_t u_t$ to dynamically adjust its temporal resolution. The projections producing B_t and C_t are bias-free.

Parallel evaluation via associative scan. The quantity that must be computed is the discrete state recurrence introduced above,

$$\xi_t = \bar{A}_t \odot \xi_{t-1} + \bar{B}_t u_t, \qquad t = 1, \ldots, L', \qquad \xi_0 = \mathbf{0},$$

The final output of the block (the intra-modal BiMamba module) is then calculated by combining the hidden state with the selective output operator using the equation $y_t = \sum_{n=1}^{N} C_{t,n}\, \xi_{t,n}$. Evaluated naively this is a loop: ξ_t cannot be formed until ξ_{t-1} exists, so the L' steps are strictly serial. The math for each step is very simple (just a multiply-add operation on a $C' \times N$ array), so calculating it one step at a time wastes GPU time on just starting the tasks and leaves the hardware mostly idle. For a sequence of $L' = 250$, this means waiting to sequentially launch 250 separate tasks for an amount of work the GPU could otherwise process almost instantly. Ultimately, the step-by-step dependency is what slows the process down, not the calculations themselves.

Two structural facts make a parallel evaluation possible. First, because A is *diagonal* in the state dimension, the update does not mix state coordinates: the vector recurrence decomposes into $B \times C' \times N$ mutually independent *scalar* recurrences, one per (batch, channel, state) triple, all sharing the same structure. Second, each such scalar recurrence is an *affine* map of the previous state. Write the step-t update as the function

$$f_t : \mathbb{R} \to \mathbb{R}, \qquad f_t(\xi) = a_t\, \xi + b_t, \qquad a_t := \bar{A}_t, \quad b_t := \bar{B}_t u_t,$$

so that unrolling from $\xi_0 = 0$ gives $\xi_t = (f_t \circ f_{t-1} \circ \cdots \circ f_1)(0)$. The state at time t is therefore nothing more than the composition of the first t update maps, evaluated at zero - and computing *every* prefix of a sequence under some combining operation is precisely what a *scan* (or prefix computation) does. The archetypal example is the prefix sum, which turns $(x_1, \ldots, x_n)$ into $(x_1,\ x_1 + x_2,\ \ldots)$; a scan generalises this to any binary operator $\oplus$.

The reason this buys anything is that a scan under an *associative* $\oplus$ need not be computed left to right. Associativity means partial results may be regrouped freely, so the sequence can be reduced pairwise in a balanced binary tree: total work stays $O(L')$, but the *sequential depth* - the length of the longest chain of dependent operations - falls from $O(L')$ to $O(\log L')$. For $L' = 250$ that is roughly 8 dependent stages instead of 250, each stage saturating the device with independent work.

Closure and the composition rule. To scan over the update maps we must be able to represent a composed map by a finite parameter set. Affine maps in one variable are closed under composition, and the family is parameterised by exactly two numbers - a multiplier and an offset - which is why the scan carries *pairs* rather than scalars. Explicitly, for $i < j$,

$$(f_j \circ f_i)(\xi) = f_j\big(f_i(\xi)\big) = a_j\big(a_i\xi + b_i\big) + b_j = \underbrace{(a_j a_i)}_{\text{new multiplier}}\ \xi + \underbrace{(a_j b_i + b_j)}_{\text{new offset}}.$$

The composite is again affine, so identifying each map with its parameter pair (a, b) induces the binary operator

$$(a_i, b_i) \oplus (a_j, b_j) := (a_j a_i,\ a_j b_i + b_j), \qquad i < j,$$

where the left operand is the earlier segment of the sequence. Intuitively the two components accumulate different things: the multiplier is the running product of decay factors, recording how much of the initial state survives, while the offset is the running total of inputs, each discounted by all the decay factors applied after it entered.

Associativity. The operator inherits associativity from function composition, but it is worth verifying directly on the parameters. Expanding the two groupings,

$$\big[(a_i, b_i) \oplus (a_j, b_j)\big] \oplus (a_k, b_k) = (a_j a_i,\ a_j b_i + b_j) \oplus (a_k, b_k) = \big(a_k a_j a_i,\ a_k a_j b_i + a_k b_j + b_k\big),$$

$$(a_i, b_i) \oplus \big[(a_j, b_j) \oplus (a_k, b_k)\big] = (a_i, b_i) \oplus (a_k a_j,\ a_k b_j + b_k) = \big(a_k a_j a_i,\ a_k a_j b_i + a_k b_j + b_k\big),$$

which agree componentwise. Together with the identity element $(1, 0)$ - the identity map $\xi \mapsto \xi$ - the parameter pairs form a monoid under $\oplus$, which is exactly the algebraic structure a scan requires. Note that $\oplus$ is *not* commutative: $(a_j a_i, a_j b_i + b_j) \neq (a_i a_j, a_i b_j + b_i)$ in general, reflecting the obvious fact that reordering the timesteps changes the trajectory. This costs us nothing, since a scan requires associativity but never commutativity.

Running the scan over $\big((a_1, b_1), \ldots, (a_{L'}, b_{L'})\big)$ yields at position t the pair representing $f_t \circ \cdots \circ f_1$; because $\xi_0 = 0$ the multiplier component annihilates and the state is read off directly from the offset component, $\xi_t = a_t \cdots a_1 \cdot 0 + (\text{offset}) = (\text{offset})$.

Implementation. JAX exposes this primitive as `jax.lax.associative_scan(fn, elems, axis)`, where `fn` implements $\oplus$ on a pair of PyTrees and is required (not merely assumed) to be associative. We supply the tuple $(\bar{A},\ \bar{B} \odot u)$, both of shape (L', B, C', N) after moving the sequence axis to the front, and scan along `axis=0`; the operator is applied elementwise, so the $B \times C' \times N$ independent recurrences are resolved simultaneously in a single call. The same routine serves the inter-modal block unchanged, where the scanned axis is the modality-channel sequence of length MC' rather than time.

Finally, the log-space parameterisation of A interacts favourably with the scan. Since $\bar{A}_t = \exp(\Delta_t A) \in (0, 1)$, the cumulative multipliers $a_t \cdots a_1$ form a decreasing product and cannot overflow, no matter how long the sequence; the worst case is underflow to zero, which is the gentle behaviour of a state that has simply forgotten its distant past. An unconstrained A permitting $|\bar{A}| > 1$ would instead make these partial products grow geometrically in the sequence length, and the tree-structured reduction would surface that divergence just as surely as a serial loop would.

4. BiMamba applies a linear projection and a residual connection to integrate bidirectional features and stabilize training:

$$u_i = h_i + \left(W_i^o \left(\frac{h_i^{\rightarrow} + h_i^{\leftarrow}}{2}\right) + b_i^o\right) \in \mathbb{R}^{L' \times C'}$$

which is the output of the intra-modal BiMamba module for the i-th modality. W_i^o is the weight matrix, and b_i^o is the bias. Observe that the residual carries the *unnormalised* h_i, not $\bar{h}_i$: the layer normalisation of step 1 conditions the block's internal computation without rescaling the identity path.

1.3.2 Inter-modal BiMamba Module

Relying solely on intra-modal modeling neglects inter-modal correlations that are obviously important for comprehensive multimodal understanding. The intra-modal stage yields M representations $u_i \in \mathbb{R}^{L' \times C'}$ of common width, and the fusion stage must let them exchange information. How that exchange is arranged - specifically, which axis the state-space model treats as its sequence - determines both what the fusion can express and whether the learned modality ordering of Section 3 has any effect at all. We first record why two natural arrangements are unsatisfactory, then give the one we adopt.

Arrangement (i): concatenate along channels, scan over time. Form

$$U^{(t)} = u_1^{(t)} \circ u_2^{(t)} \circ \cdots \circ u_M^{(t)} \in \mathbb{R}^{MC'},$$

where $\circ$ denotes concatenation and $u_i^{(t)} \in \mathbb{R}^{C'}$ is the i-th modality's representation at time t, and apply the BiMamba block along the time axis with feature width $D = MC'$. Modalities then interact only through the block's dense projections $W \in \mathbb{R}^{MC' \times MC'}$, which mix all channels of all modalities at every time step. The fusion is expressive but structurally unconstrained, and it has two problems.

First, any custom reordering of the input modalities becomes useless. Because the reordering operation happens right before a dense layer (a linear weight matrix), the network simply absorbs the fixed rearrangement directly into its weight calculations without changing the model's capabilities. The only benefit comes from making the reordering dynamic (input-dependent), but even then, it creates a soft blend of modalities rather than a true sequential order. This defeats the original goal of learning an ordering, which relies on processing the modalities step-by-step.

Formally, let $P \in \mathbb{R}^{M \times M}$ be the reordering matrix of Section 3.2 and let $P \otimes I_{C'}$ denote its Kronecker product with the $C' \times C'$ identity - the block matrix that permutes whole modality blocks while acting as the identity within each block's channels. Reordering before the dense projection then gives $W(P \otimes I_{C'})U^{(t)} = W'U^{(t)}$ with $W' := W(P \otimes I_{C'})$, so for any fixed P the reordered model computes exactly what the unreordered model computes with weights W': the two are the same model class, and the permutation is unlearnable because it is already absorbed.

Second, the parameter count grows quadratically with the number of modalities (M). Combining all modalities into a single wide feature dimension causes the fusion layer to require roughly M^2 times as many parameters. This makes the fusion stage take up most of the network's capacity, which easily leads to severe overfitting on smaller datasets containing only a few dozen sessions.

Arrangement (ii): concatenate along channels, scan over the concatenated axis. A second approach treats the combined index as the sequence, scanning over all MC' positions so each modality forms a block of C' steps. This fails due to an information bottleneck, which becomes clear when we unroll the recurrence over the scan index k:

$$\xi_k = \bar{A}_k \, \xi_{k-1} + b_k, \qquad b_k := \bar{B}_k s_k, \qquad \xi_0 = \mathbf{0},$$

where k is the position, s_k is the input, ξ_k is the hidden state, $\bar{A}_k$ is the decay factor, and $\bar{B}_k$

is the input operator. Unrolling this step-by-step gives:

$$\begin{aligned}
\xi_1 &= \bar{A}_1 \cdot \mathbf{0} + b_1 = b_1, \\
\xi_2 &= \bar{A}_2 \xi_1 + b_2 = \bar{A}_2 b_1 + b_2, \\
\xi_3 &= \bar{A}_3 \xi_2 + b_3 = \bar{A}_3 \bar{A}_2 b_1 + \bar{A}_3 b_2 + b_3, \\
&\vdots \\
\xi_k &= \sum_{k'=1}^{k} \left(\prod_{l=k'+1}^{k} \bar{A}_l \right) b_{k'},
\end{aligned}$$

with an empty product equaling 1. The term $\prod_{l=k'+1}^{k} \bar{A}_l$ is the weight applied to past inputs; to calculate this, you must multiply the retention factors for every step between the past input k' and the current position k. At the start of training, the model's ability to dynamically change this factor based on input is weak, so the decay factor stays close to a constant baseline, $\bar{A}$. Because it is roughly constant, multiplying it for every step in the gap simplifies to $\bar{A}$ raised to the power of the distance between positions:

$$\prod_{l=k'+1}^{k} \bar{A}_l \approx \bar{A}^{\,k-k'}$$

Under this arrangement, one modality is separated from the next by C' steps, so the information passed between them shrinks by a factor of $\bar{A}^{\,C'}$. Since $\bar{A}$ is a fraction, raising it to a large power like C' shrinks the surviving information to practically zero. To successfully pass information across this gap, the network would have to push $\bar{A}$ incredibly close to 1. However, doing so stops the memory from fading, causing the block to lose its ability to distinguish different positions along the sequence. Furthermore, the gradient signal needed to increase $\bar{A}$ is blocked by this exact same bottleneck.

Arrangement (iii), adopted: scan over the modality axis. Both of the previous problems are solved if we treat the list of modalities itself as the sequence. Instead of a single modality taking up C' steps, it now takes up exactly one position. First, we stack the dynamically reordered modality outputs into a new shape:

$$\tilde{\mathcal{U}} \in \mathbb{R}^{M \times L' \times C'}, \qquad \tilde{\mathcal{U}}_i = \tilde{u}_i,$$

where $\tilde{u}_i = \sum_{j=1}^{M} P_{ij} u_j$. At every individual time step t, the model processes a sequence of M vectors, $\big(\tilde{u}_1^{(t)}, \ldots, \tilde{u}_M^{(t)}\big)$, each with a size of C'. The BiMamba block is then applied directly across this sequence of modalities:

$$H^{(t)} = \mathrm{BiMamba}_{\mathrm{modality}}\big(\tilde{u}_1^{(t)}, \ldots, \tilde{u}_M^{(t)}\big) \in \mathbb{R}^{M \times C'}, \qquad t = 1, \ldots, L'.$$

This is calculated independently for each time step using the same shared weights. The job of understanding time is left entirely to a previous part of the network.

Because modalities are now sitting directly next to each other in the sequence, the memory decay between them is only $\bar{A}$ instead of the massive $\bar{A}^{\,C'}$ bottleneck. This allows information to flow easily between them, no matter what decay rate the model learns. When scanning forward, modality i gathers context from the modalities placed before it, $\{1, \ldots, i-1\}$. When scanning backward, it gathers context from the modalities placed after it, $\{M, \ldots, i+1\}$. Averaging these two directions guarantees that every modality can "see" all the others.

This setup fixes the core issues in two major ways:

- **The sorting mechanism works:** Because the fusion process is now sensitive to order, the learned permutation actively decides which modalities give context to which. This effect can no longer be bypassed or absorbed by standard network weights.
- **Efficiency gains:** The feature width shrinks to C', which drops the parameter cost from a heavy $\Theta(M^2C'^2)$ down to just $\Theta(C'^2)$. Additionally, the depth of the dependent math operations (the associative scan) drops from $\lceil \log_2 L' \rceil$ to just $\lceil \log_2 M \rceil$, running on a much smaller state.

Finally, the output is stitched back together side-by-side to match the expected format for the final prediction heads:

$$H^{(t)} = H_1^{(t)} \circ \cdots \circ H_M^{(t)} \in \mathbb{R}^{MC'}$$

which returns the data to its original shape of $H \in \mathbb{R}^{L' \times MC'}$.

1.4 Differential Entropy and the Failure of Gaussian Assumptions

To adapt the entropy-based selection mechanism of BiMamba-TTA for continuous engagement regression, we transition from Shannon entropy to **differential entropy**. For a continuous random variable y with probability density $p(y)$, the differential entropy is defined as:

$$H(y) = -\int_{\mathcal{Y}} p(y) \ln p(y)\, dy$$

While a Gaussian assumption $p(y|x) = \mathcal{N}(\mu, \sigma^2)$ simplifies this to $H(y) = \frac{1}{2}\ln(2\pi e\sigma^2)$, it is fundamentally inappropriate for engagement modeling. Engagement scores are strictly bounded within the interval $[0, 1]$. A Gaussian distribution assigns non-zero probability mass to values in $(-\infty, 0)$ and $(1, \infty)$, leading to "probability leakage" and poor calibration near the boundaries.

1.5 Beta Regression Framework

To respect the support $\mathcal{Y} \in [0, 1]$, we model the predictive distribution using a **Beta distribution**:

$$p(y|\alpha, \beta) = \frac{y^{\alpha-1}(1-y)^{\beta-1}}{B(\alpha, \beta)}, \quad B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha+\beta)}$$

Head architecture. Each Beta head is a two-layer perceptron rather than a bare linear readout. Let $r \in \mathbb{R}^{L' \times d_{\text{in}}}$ denote the representation entering the head, where d_{in} is its channel width. The head first maps r through one hidden layer,

$$v = \sigma\big(W^{(1)} r + b^{(1)}\big) \in \mathbb{R}^{L' \times d_h},$$

where the superscript (1) indexes the first (and only) hidden layer: $W^{(1)} \in \mathbb{R}^{d_h \times d_{\text{in}}}$ is its weight matrix and $b^{(1)} \in \mathbb{R}^{d_h}$ its bias, σ is the SiLU activation, and d_h is the head width. The hidden activation v is then read out by two separate linear layers, one per shape parameter:

$$z_\alpha = W^{(\alpha)} v + b^{(\alpha)}, \qquad z_\beta = W^{(\beta)} v + b^{(\beta)}, \qquad W^{(\alpha)}, W^{(\beta)} \in \mathbb{R}^{1 \times d_h}$$

Both read-outs consume the *same* v: the hidden layer extracts one shared intermediate feature from which the two shape parameters are then predicted independently. This lets α and β draw on common structure - both describe the same predictive distribution - while still being free to move in opposite directions, which is what allows the head to represent "high mean, low variance" and "high mean, high variance" as distinct outputs.

The network outputs the shape parameters via a softplus activation: $\alpha = \text{softplus}(z_\alpha) + 1$ and $\beta = \text{softplus}(z_\beta) + 1$. The $+1$ offset ensures the distribution remains unimodal ($\alpha, \beta > 1$), which also bounds the density away from the divergent behaviour that arises as $\alpha, \beta \to 0^+$.

The predictive mean $\hat{\mu}$ and aleatoric uncertainty $\mathrm{Var}[y]$ are:

$$\hat{\mu} = \frac{\alpha}{\alpha + \beta}, \quad \mathrm{Var}[y] = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}$$

The Beta variance is naturally boundary-aware, satisfying:

$$\lim_{\hat{\mu}\to 0} \mathrm{Var}[y] = 0 \quad \text{and} \quad \lim_{\hat{\mu}\to 1} \mathrm{Var}[y] = 0$$

This provides a principled measure of confidence that scales with the proximity to the interval limits.

Target clipping. Taking the logarithm of the Beta density above gives

$$\ln p(y \mid \alpha, \beta) = (\alpha - 1)\ln y + (\beta - 1)\ln(1 - y) - \ln B(\alpha, \beta),$$

so the log-likelihood contains the terms $\ln y$ and $\ln(1-y)$ explicitly. Both diverge at the endpoints of the support: $\ln y \to -\infty$ as $y \to 0^+$, and $\ln(1 - y) \to -\infty$ as $y \to 1^-$. The engagement annotations do attain exactly 0 and 1, at which points the expression is undefined and evaluates to `-inf` or `NaN` in floating-point arithmetic - a single such frame is enough to corrupt the gradient of the entire batch. Targets are therefore clipped to $[\varepsilon,\, 1-\varepsilon]$ with $\varepsilon = 10^{-6}$ before the likelihood is evaluated. This is a numerical safeguard only: it moves an affected target by 10^{-6} on a scale of $[0, 1]$, which is four orders of magnitude below the resolution at which human annotators can meaningfully distinguish engagement levels.

1.6 Test-Time Adaptation (TTA) and Adaptive Thresholds

Following the differential-entropy principle of the original BiMamba-TTA paper [1], we replace Shannon entropy with a regression-compatible uncertainty proxy.

For the Beta predictive distribution $p(y \mid x) = \mathrm{Beta}(\hat{\alpha}(x), \hat{\beta}(x))$ that respects the support $y \in [0, 1]$, we define

$$U(x) := \mathrm{Var}[y] = \frac{\hat{\alpha}(x)\hat{\beta}(x)}{(\hat{\alpha}(x) + \hat{\beta}(x))^2(\hat{\alpha}(x) + \hat{\beta}(x) + 1)}$$

What $U(x)$ is, intuitively. In the original BiMamba-TTA formulation [1], $U(x)$ is the model's own admission of how unsure it is about the sample x. Test data carries no labels, so the model cannot know which of its predictions are correct; $U(x)$ is the next best thing, a self-assessment used to decide which unlabelled frames are trustworthy enough to learn from. Low $U(x)$ means the predictive distribution is sharply peaked - the model is committing to an answer - and such frames make safe pseudo-labels. High $U(x)$ means the distribution is spread out and the prediction is close to a guess; adapting on those would teach the model its own noise.

Why variance. For classification, $U(x)$ is the Shannon entropy of the softmax output, a direct measure of how spread the predicted distribution is. Regression has no softmax, so the natural generalisation is the differential entropy $H(p)$ of the predictive density - but for a Beta distribution $H(p)$ requires the log-Beta function and two digamma evaluations per frame, which is the transcendental cost we are trying to avoid in a real-time streaming filter.

The way out is to notice what the filter actually does with $U(x)$. It compares $U(x)$ against a percentile of recently observed values (below), so the only property that matters is the *ordering* it induces over samples. Any function that is a strictly monotonic transform of $H(p)$ therefore selects exactly the same frames as $H(p)$ itself, at whatever cost we like. This reframes the problem from "compute the entropy" to "find the cheapest quantity that ranks samples the

same way", and the predictive variance is the obvious first candidate: it is the standard measure of spread, it is already available in closed form from the shape parameters the network emits, and it costs four arithmetic operations. The remainder of this subsection verifies that this candidate does in fact preserve the ordering.

1.6.1 Why Predictive Variance is a Strictly Monotonic Proxy for Differential Entropy

This choice is justified because variance is a *strictly monotonic* function of the differential entropy $H(p)$ of the Beta distribution. To see this explicitly, re-parameterise the Beta distribution by its mean and concentration:

$$\mu = \frac{\hat{\alpha}}{\hat{\alpha} + \hat{\beta}}, \qquad \kappa = \hat{\alpha} + \hat{\beta}.$$

Substituting these parameters yields the simple expression

$$U(x) = \frac{\mu(1-\mu)}{\kappa + 1}.$$

For any fixed mean $\mu \in (0, 1)$, the partial derivative

$$\frac{\partial U}{\partial \kappa} = -\frac{\mu(1-\mu)}{(\kappa+1)^2} < 0$$

shows that variance is strictly decreasing in κ: larger concentration κ produces a tighter distribution and therefore smaller variance.

The differential entropy of the Beta distribution has the closed-form expression

$$H(p) = \ln B(\hat{\alpha}, \hat{\beta}) - (\hat{\alpha} - 1)\psi(\hat{\alpha}) - (\hat{\beta} - 1)\psi(\hat{\beta}) + (\hat{\alpha} + \hat{\beta} - 2)\psi(\hat{\alpha} + \hat{\beta}),$$

where $B(\cdot, \cdot)$ is the Beta function and $\psi(\cdot)$ is the digamma function. Substituting the same re-parameterisation $\hat{\alpha} = \mu\kappa$, $\hat{\beta} = (1-\mu)\kappa$ and taking the derivative with respect to κ likewise yields

$$\frac{\partial H}{\partial \kappa} < 0$$

for any fixed $\mu \in (0, 1)$. Thus both $U(x)$ and $H(p)$ are strictly decreasing functions of κ: larger concentration (tighter distribution) produces both smaller variance *and* smaller differential entropy. Consequently, the ranking of samples by uncertainty is identical whether one uses the full (expensive) differential entropy expression or simply the predictive variance.

Variance is numerically stable because its formula consists only of basic arithmetic operations (no logarithms, no digamma or Gamma functions, and no risk of underflow when $\hat{\alpha}$ or $\hat{\beta}$ become large). It is boundary-aware: $\lim_{\mu \to 0^+} U = 0$ and $\lim_{\mu \to 1^-} U = 0$, which matches the intuition that a prediction that is certain to be near the extremes of engagement (fully disengaged or fully engaged) should carry low uncertainty. Finally, variance is already required for the Beta negative log-likelihood (Beta NLL) loss used during training:

$$\mathcal{L}_{\text{NLL}}(x, y) = -[(\hat{\alpha} - 1)\ln y + (\hat{\beta} - 1)\ln(1 - y) - \ln B(\hat{\alpha}, \hat{\beta})].$$

Because the variance appears directly in the predictive distribution (and therefore in every forward pass), it incurs no extra computation at inference time. All these properties make variance the natural, cheap uncertainty proxy for regression.

A sample is selected for the TTA backward pass if the multimodal prediction is confident (low uncertainty) but the unimodal branches are uncertain (high uncertainty), i.e.,

$$S(x) = \mathbb{I}\Big\{U_{\text{multi}}(x) \le \gamma_m(t) \text{ AND } \sum_{i=1}^{M} w_i U_i(x) \ge \gamma_u(t)\Big\}.$$

To handle non-stationary streaming data (i.e., test-time engagement signals that arrive continuously frame-by-frame and whose statistical distribution drifts over time due to fatigue, cultural differences, sensor drift, etc.), the thresholds $\gamma(t)$ are updated adaptively using a moving window of the most recent K samples:

$$\gamma(t) = \text{Percentile}\big(\mathcal{U}_t,\, p\big), \quad \mathcal{U}_t = \big\{U(\tau) \mid \tau \in [t-K, t]\big\},$$

where p is the percentile rank (e.g., $p = 0.8$) that defines the "confidence" cutoff relative to the recent noise floor of the data stream. Here "non-stationary streaming data" means the incoming test distribution is not fixed but changes gradually; the moving window $\mathcal{U}_t$ tracks this drift in real time.

The percentile rank p makes the threshold *relative rather than absolute*. A fixed cutoff such as $U(x) \le 0.02$ would be meaningless across domains: a model that is uniformly less certain on Japanese sessions than on English ones would have every frame rejected on one corpus and every frame accepted on the other, purely because its overall uncertainty level shifted. By setting $\gamma(t)$ to the p-th percentile of the last K observed uncertainties, "confident" is redefined at each moment to mean *confident by the standards of what this model is currently producing on this stream*, and the filter keeps selecting a stable fraction of frames regardless of the absolute uncertainty level. Percentiles are used rather than the mean and standard deviation because they are order statistics and therefore insensitive to the occasional extreme value - a few corrupted frames shift a percentile barely at all, whereas they can move a mean arbitrarily far.

1.7 Uncertainty Estimation Methods

Uncertainty is formally quantified as the variance of the predictive distribution. We describe three complementary methods that together capture both aleatoric and epistemic uncertainty. All three are expressed in terms of the Beta predictive distribution $p(y \mid x) = \text{Beta}(\hat{\alpha}(x), \hat{\beta}(x))$. Of these, only the first is realised in the current implementation; methods 2 and 3 require stochastic regularisation that the present architecture does not contain (see the remark at the end of this subsection).

1. **Parametric Beta (Aleatoric Uncertainty):** This is the direct predictive variance obtained from the dual-head Beta output:

   $$U(x) = \text{Var}[y] = \frac{\hat{\alpha}(x)\hat{\beta}(x)}{(\hat{\alpha}(x) + \hat{\beta}(x))^2(\hat{\alpha}(x) + \hat{\beta}(x) + 1)}.$$

   Because the variance is computed analytically from the learned shape parameters, it is the fastest and default choice for the TTA filter, and it is the quantity actually used by the implemented selection rule $S(x)$.

2. **Monte Carlo Dropout (Epistemic Uncertainty):** We keep dropout active at inference and draw T stochastic forward passes (typically $T = 10$–20). For each pass t we obtain a Beta distribution with parameters $(\hat{\alpha}^{(t)}, \hat{\beta}^{(t)})$ and corresponding mean $\hat{\mu}^{(t)} = \dfrac{\hat{\alpha}^{(t)}}{\hat{\alpha}^{(t)} + \hat{\beta}^{(t)}}$. The epistemic uncertainty is the variance of these means:

   $$U_{\text{epi}}(x) = \frac{1}{T}\sum_{t=1}^{T}\big(\hat{\mu}^{(t)}(x) - \bar{\mu}(x)\big)^2,$$

where $\bar{\mu}(x) = \frac{1}{T}\sum_{t=1}^{T}\hat{\mu}^{(t)}(x)$. This term quantifies the model's "lack of knowledge" (how sensitive the prediction is to random weight masks) and is especially useful under distribution shift (NoXi → NoXi+J).

3. **Total Uncertainty (Hybrid):** Using the law of total variance, the full predictive uncertainty decomposes as

$$U_{\text{total}}(x) = \underbrace{\frac{1}{T}\sum_{t=1}^{T}\operatorname{Var}\big[\operatorname{Beta}(\hat{\alpha}^{(t)},\hat{\beta}^{(t)})\big]}_{\text{aleatoric (data)}} + \underbrace{\frac{1}{T}\sum_{t=1}^{T}\big(\hat{\mu}^{(t)}(x)-\bar{\mu}(x)\big)^2}_{\text{epistemic (model)}}.$$

   The decomposition is mainly used as a diagnostic to monitor whether the shift is mostly epistemic.

Note that the symbol $U(x)$ in the TTA filter always refers to one of the three quantities above (most often the parametric Beta variance for speed or the hybrid total variance when epistemic information is needed). The law of total variance guarantees that U_{total} is the correct variance of the mixture distribution induced by MC dropout.

Implementation status. The current architecture contains no dropout layers, so methods 2 and 3 cannot be evaluated as written: with deterministic weights, every one of the T forward passes returns the identical Beta distribution and $U_{\text{epi}} \equiv 0$. Realising them requires inserting dropout into the Beta heads (and optionally the BiMamba projections) and retaining it at inference. Until then, methods 2 and 3 should be read as a planned extension rather than a component of the reported system.

1.8 Training Objective

The likelihood we train on and the metric we are scored by ask different questions. The Beta negative log-likelihood looks at one frame at a time and asks whether the predicted distribution is right there. CCC looks at all frames together and asks whether the predictions rise and fall in the same pattern as the targets. In theory, a model can place a sensible distribution on every frame and still order the frames badly. We therefore add three terms to the likelihood,

$$\mathcal{L} = \underbrace{\frac{1}{|\mathcal{R}|}\sum_{r\in\mathcal{R}}\mathcal{L}^{w}_{\text{NLL}}\big(\alpha^r,\beta^r,y^r\big)}_{\text{per-frame fit}} + \underbrace{\lambda_c\big(1-\widehat{\text{CCC}}\big)}_{\text{ordering}} + \underbrace{\frac{\lambda_u}{M}\sum_{i=1}^{M}\mathcal{L}^{w}_{\text{NLL}}\big(\alpha_i,\beta_i,y^{r(i)}\big)}_{\text{per-modality supervision}} + \underbrace{\lambda_f\,\mathcal{L}_{\text{fair}}}_{\text{fairness}},$$

where $r(i)$ denotes the role of stream i, and $\lambda_c, \lambda_u, \lambda_f \geq 0$ set how much each term counts.

Reweighting the likelihood. Describe a Beta density by its mean $\mu = \alpha/(\alpha+\beta)$ and its precision $\kappa = \alpha+\beta$, so $\alpha = \mu\kappa$ and $\beta = (1-\mu)\kappa$; large κ means a narrow, confident distribution. Differentiating $\ln p(y) = (\alpha-1)\ln y + (\beta-1)\ln(1-y) - \ln B(\alpha,\beta)$ with respect to μ, with $\partial\alpha/\partial\mu = \kappa$ and $\partial\beta/\partial\mu = -\kappa$, and using $\ln B(\alpha,\beta) = \ln\Gamma(\alpha) + \ln\Gamma(\beta) - \ln\Gamma(\alpha+\beta)$ so that $\partial \ln B/\partial\alpha = \psi(\alpha) - \psi(\alpha+\beta)$ and $\partial \ln B/\partial\beta = \psi(\beta) - \psi(\alpha+\beta)$, the $\psi(\alpha+\beta)$ terms cancel and leave

$$\frac{\partial(-\ln p)}{\partial\mu} = \kappa\left[\ln\frac{1-y}{y} + \psi(\mu\kappa) - \psi\big((1-\mu)\kappa\big)\right] \;\propto\; \kappa.$$

The correction the mean receives is thus multiplied by the model's own confidence, so frames it is already sure about push hardest and frames it is unsure about barely move it - the opposite of what we want, since the uncertain frames are the ones with something left to learn. This is a known failure of likelihood training with predicted uncertainty [21]. We fix it by scaling each

frame's loss by $w_t = \kappa_t^{-\beta_w}$, giving $\mathcal{L}^w_{\mathrm{NLL}} = \sum_t w_t \mathcal{L}_{\mathrm{NLL},t} / \sum_t w_t$ and a correction proportional to $\kappa^{1-\beta_w}$; $\beta_w = 0$ leaves the likelihood untouched and $\beta_w = 1$ removes the confidence factor completely, and we use $\beta_w = 0.5$. Crucially w_t is computed from the current α_t, β_t but treated as a fixed number when derivatives are taken, so no gradient flows back through it. Were it differentiable, the model could shrink the loss by inflating κ to make w_t small, rather than by predicting better - the weight is there to rebalance the update, not to become another thing to optimise.

Ordering term. $\widehat{\mathrm{CCC}}$ is the sample concordance coefficient of Section 4.1 computed on the predicted means and the targets of the current minibatch, pooled across both roles and all frames in it. Every ingredient - the two means, the two variances, the covariance - is a smooth function of the predictions, so the term can be differentiated, and since $\mathrm{CCC} = \rho\, C_b$, minimising $1 - \widehat{\mathrm{CCC}}$ pushes on exactly the factor the likelihood leaves loose. Two limitations follow from estimating it on a batch. It is biased when the batch is small, because the same frames supply both the moments and the ratio built from them; and it reflects the pooled quantity we are scored on only if the batch spans several sessions, since a batch from one session measures ordering within that session alone. Batches are accordingly drawn from windows of several sessions.

Per-modality supervision. Each auxiliary head is trained on the target of the role whose stream it reads. This keeps every modality branch predictive in its own right instead of drifting into whatever representation happens to suit the fusion stage, and it supplies the calibrated per-modality distributions that test-time adaptation (Section 2.6) compares against each other. Dividing by M keeps this contribution from growing with the number of streams: at $\lambda_u = 0.5$ and M streams the undivided sum would outweigh the multimodal term by $\lambda_u M$, which for the core configuration is a factor of four - so most of the gradient would come from heads that are thrown away at evaluation time.

Fairness term. The Conditional Demographic Disparity of Section 4.2 is built from differences of group averages inside strata of the ground truth, and every step is differentiable, so it can be trained against rather than only measured. Splitting the batch into n_b quantile bins $\mathcal{B}_1, \ldots, \mathcal{B}_{n_b}$ of the target, and writing $\overline{\hat{\mu}}_b^{\,A}$ and $\overline{\hat{\mu}}_b^{\,B}$ for each group's mean prediction inside bin b, we penalise

$$\mathcal{L}_{\mathrm{fair}} = \frac{\sum_{b=1}^{n_b} |\mathcal{B}_b| \left(\overline{\hat{\mu}}_b^{\,A} - \overline{\hat{\mu}}_b^{\,B} \right)^2}{\sum_{b=1}^{n_b} |\mathcal{B}_b|},$$

with bins missing either group left out of both sums. Squaring removes the sign, so the term is smallest exactly when the two groups get the same prediction at the same true engagement, and weighting by bin size matches how the metric is estimated at evaluation. Note that the penalty is identically zero on a batch containing only one group, so batches must mix groups for it to do anything.

1.9 Test-Time Loss Derivations

The TTA objective enforces consistency between the unimodal student heads and the multimodal teacher while still supervising the students toward the multimodal prediction. We define the **Mutual Information Sharing (MIS) loss** as the KL divergence that pushes each unimodal Beta distribution toward the multimodal one:

$$\mathcal{L}_{\mathrm{mis}} = \sum_{i=1}^{M} D_{KL}\big(\mathrm{Beta}(\alpha_i, \beta_i) \parallel \mathrm{Beta}(\alpha_{\mathrm{multi}}, \beta_{\mathrm{multi}})\big),$$

where the closed-form Beta KL divergence, with $B_i := B(\alpha_i, \beta_i)$, is:

$$D_{\mathrm{KL}}(B_1 \parallel B_2) = \ln \frac{B(\alpha_2, \beta_2)}{B(\alpha_1, \beta_1)} + (\alpha_1 - \alpha_2)\psi(\alpha_1) + (\beta_1 - \beta_2)\psi(\beta_1) + (\alpha_2 - \alpha_1 + \beta_2 - \beta_1)\psi(\alpha_1 + \beta_1) \quad (1)$$

The final TTA loss combines the MIS consistency term with a supervision term that treats the multimodal mean as a pseudo-target for each unimodal head:

$$\mathcal{L}_{TTA} = \mathcal{L}_{\mathrm{mis}} + \lambda \sum_{i=1}^{M} \mathrm{NLL}_{\mathrm{Beta}}\big(\hat{\mu}_{\mathrm{multi}},\, \alpha_i, \beta_i\big).$$

Here $\lambda > 0$ is a hyperparameter that controls the relative importance of the two terms: larger λ emphasises direct supervision from the multimodal teacher, while smaller λ puts more weight on cross-modal consistency. The term $\mathrm{NLL}_{\mathrm{Beta}}(\hat{\mu}_{\mathrm{multi}}, \alpha_i, \beta_i)$ is the Beta negative log-likelihood evaluated at the multimodal mean $\hat{\mu}_{\mathrm{multi}}$ (treated as a pseudo-label) with the unimodal shape parameters.

1.10 Surgical Fine-Tuning and Variance Starvation

We only update specific surgical layers to prevent catastrophic forgetting:

- **BatchNorm layers:** These encode per-channel activation statistics, allowing for immediate adaptation to domain shifts. In EngageNet these are the BN stages inside every InitEncoder.
- **First convolution in InitEncoder:** Re-calibrates low-level feature extraction (e.g., sensor noise) before temporal/modal mixing.
- **First FC layer in Inter-modal BiMamba:** Recalibrates cross-modal fusion (e.g., adjusting the weight of audio vs. visual cues for different cultures). Concretely this is the first dense projection inside the cross-modal BiMamba block.

Gradients are masked so that all remaining parameters receive exactly zero update.

This surgical approach mitigates the **Variance Starvation Problem (VSP)**. In VSP, a model minimises the Beta NLL by artificially inflating $\mathrm{Var}[y]$ (i.e., reducing the concentration $\kappa = \hat{\alpha} + \hat{\beta}$) on hard samples; the gradient to the mean head then vanishes because the loss term becomes insensitive to prediction error. By freezing the majority of the backbone and updating only the listed surgical layers, we force the model to improve genuine feature alignment rather than using uncertainty as a "slack variable" to bypass difficult data.

1.11 Architectural Justification for the Beta Regression Framework

To model bounded continuous engagement levels $y \in [0, 1]$ alongside reliable predictive uncertainty, several alternative statistical formulations could be considered: the Kumaraswamy distribution, logit-normal regression, and distribution-free conformalized regression. In this section, we mathematically demonstrate why the Beta distribution is uniquely suited to meet the real-time constraints, closed-form requirement, and test-time adaptation (TTA) mechanics of EngageNet.

Comparison with the Kumaraswamy Distribution

The Kumaraswamy distribution [8] is often preferred over the Beta distribution in standard deep learning tasks [9] because its probability density function and cumulative distribution function rely strictly on elementary algebraic powers, avoiding transcendental functions. For

shape parameters $a, b > 0$, its CDF is $F(x; a, b) = 1 - (1 - x^a)^b$. This formulation simplifies the calculation of the Negative Log-Likelihood (NLL) during training and avoids computing digamma and trigamma functions during backpropagation, circumventing the gradient instabilities that cause the Variance Starvation Problem (VSP).

However, EngageNet's TTA selection filter $S(x)$ evaluates the predictive variance $U(x)$ at *inference time* on every incoming frame of a non-stationary data stream. The n-th raw moment of a Kumaraswamy random variable Y is given by:

$$\mathbb{E}[Y^n] = b \cdot B\left(1 + \frac{n}{a}, b\right) = \frac{b\,\Gamma(1 + n/a)\,\Gamma(b)}{\Gamma(1 + n/a + b)}$$

where $B(\cdot, \cdot)$ is the transcendental Beta function and $\Gamma(\cdot)$ is the Gamma function. Consequently, the predictive variance of the Kumaraswamy distribution is expressed as:

$$\mathrm{Var}[Y] = b \cdot B\left(1 + \frac{2}{a}, b\right) - \left[b \cdot B\left(1 + \frac{1}{a}, b\right)\right]^2$$

If EngageNet adopted the Kumaraswamy distribution, the model would be forced to compute multiple expensive Gamma and Beta functions during real-time streaming inference to evaluate the TTA filter. In contrast, the Beta distribution shifts this transcendental computational burden entirely to the offline training phase via the polygamma functions in the backward pass. Its inference-time predictive variance $U(x)$ is computed via lightweight, pure algebraic operations:

$$U(x) = \frac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}$$

Furthermore, proving a strictly monotonic relationship between the variance $\mathrm{Var}[Y]$ and the differential entropy $H(Y)$ under arbitrary parameter updates is mathematically intractable for the Kumaraswamy distribution, whereas the Beta distribution cleanly satisfies $\frac{\partial U}{\partial \kappa} < 0$ and $\frac{\partial H}{\partial \kappa} < 0$ under the precision mapping $\kappa = \alpha + \beta$.

Comparison with Logit-Normal Regression

Logit-normal regression maps the bounded domain $y \in [0, 1]$ to an unbounded latent domain $z \in \mathbb{R}$ using the logit transformation, assuming the latent variable follows a Gaussian distribution:

$$z = \mathrm{logit}(y) = \ln\left(\frac{y}{1 - y}\right) \sim \mathcal{N}(\mu, \sigma^2)$$

While the latent Gaussian log-likelihood is highly stable to optimize during training, the moments of the resulting logit-normal variable do not exist in closed form. The true predictive mean and variance must be resolved via numerical integration:

$$\mathbb{E}[Y] = \int_0^1 \frac{1}{y(1 - y)\sigma\sqrt{2\pi}} \exp\left(-\frac{(\mathrm{logit}(y) - \mu)^2}{2\sigma^2}\right) \cdot y\, dy$$

Performing numerical integration over N categories across multiple unimodal branches for every incoming frame introduces an insurmountable latency bottleneck during real-time test-time adaptation.

Crucially, EngageNet utilizes an inter-modal information sharing loss ($\mathcal{L}_{mis}$) that minimizes the Kullback-Leibler (KL) divergence between the unimodal predictions and the fused network prediction. While computing the KL divergence between two latent Gaussians in logit space is trivial, minimizing it optimizes a distorted geometry that stretches gradients to infinity near the boundaries (0 and 1). Computing the exact KL divergence between two true logit-normal densities within the probability domain $[0, 1]$ lacks an analytical closed-form expression [7]. Because the Beta distribution possesses a known analytical KL divergence 1, it provides a stable, exact, and geometry-preserving target for cross-modal alignment.

Comparison with Conformalized Regression

Conformalized regression (or Conformal Prediction) [10] offers a distribution-free, non-parametric framework that provides a valid prediction interval $[\hat{y}-q, \hat{y}+q]$ satisfying a user-defined coverage guarantee $1-\alpha$. While mathematically robust, conformal prediction fundamentally requires the data stream to satisfy the property of exchangeability.

In streaming test-time adaptation settings, the user exhibits non-stationary distribution shifts (e.g., sensor degradation, fluctuating lighting conditions, or progressive fatigue). This explicitly violates the exchangeability assumption, causing the empirical quantile threshold q computed on a static source calibration set to lose its strict coverage guarantees. While online adaptive variants (such as Adaptive Conformal Inference) [11] can dynamically scale the interval width based on sequential coverage errors, they operate as reactive heuristic controllers rather than generative models of the target density.

Most importantly, conformal prediction produces a discrete boundary region rather than a smooth, parametric probability density function. Because it is non-differentiable, it cannot supply a gradient operator to backpropagate local self-supervised likelihood updates ($\mathcal{L}_{TTA}$) during a single-frame backward pass. It also cannot participate in information-theoretic objectives like the mutual information sharing loss ($\mathcal{L}_{mis}$). Thus, while conformal regression is excellent for post-hoc uncertainty visualization, it cannot serve as the active functional engine that drives online weight updates within an adaptive neural network architecture.

2 Permutation-Equivariant Inter-modal Fusion

2.1 The Order-Sensitivity Problem

Recall from Section 1.3.2 that the inter-modal BiMamba module treats modality identity as its sequence axis, processing at each time step the length-M sequence

$$\big(u_1^{(t)}, u_2^{(t)}, \dots, u_M^{(t)}\big), \qquad u_i^{(t)} \in \mathbb{R}^{C'},$$

with each modality contributing exactly one position.

The ordering of these blocks - audio before video, or video before pose - is chosen arbitrarily by the practitioner; no prior knowledge assigns a canonical rank to distinct measurement modalities. We show that this arbitrariness introduces a structural bias, because selective state-space models are inherently order-sensitive.

Let $\xi_k \in \mathbb{R}^{L'}$ denote the SSM hidden state at sequence position k, and let $s_k \in \mathbb{R}^{L'}$ be the input at that position (corresponding to the channel block of the k-th modality in m). The Mamba selective SSM [2] defines the recurrence

$$\xi_k = \bar{A}_k(s_k)\,\xi_{k-1} + \bar{B}_k(s_k)\,s_k, \qquad y_k = C_k(s_k)\,\xi_k,$$

where $\xi_0 = \mathbf{0}$ and $\bar{A}_k, \bar{B}_k, C_k$ are input-dependent operators instantiated via the selection mechanism.

Proposition. For any SSM with non-trivial state transition (i.e. $\bar{A} \neq I$), the output sequence is not invariant under permutations of the input.

Proof. It suffices to consider $M = 2$ and a linear, non-selective SSM with constant operators $\bar{A}, \bar{B}, C$. Under the ordering (u_1, u_2):

$$\xi_1 = \bar{B}\,u_1, \qquad y_2 = C\big(\bar{A}\bar{B}\,u_1 + \bar{B}\,u_2\big)\,.$$

Under the swapped ordering (u_2, u_1):

$$\xi_1' = \bar{B}\, u_2, \qquad y_2' = C\big(\bar{A}\bar{B}\, u_2 + \bar{B}\, u_1\big)\,.$$

The difference is

$$y_2 - y_2' = C\bar{B}(\bar{A} - I)(u_1 - u_2).$$

This vanishes for all $u_1 \neq u_2$ only if $\bar{A} = I$, which degenerates the SSM to a memoryless map and eliminates all temporal memory - contradicting the purpose of state-space modelling. In the selective Mamba SSM, the input-dependent operators $\bar{A}_k(s_k)$ and $\bar{B}_k(s_k)$ introduce additional asymmetric cross-terms that further break permutation symmetry. □

Bidirectionality alone does not resolve the problem. In the backward pass, $\mathrm{Rev}_t(\cdot)$ reverses along the *temporal* axis L', not along the modality-sequence axis MC'. The backward SSM still processes modality blocks in the order $M, M-1, \ldots, 1$; each modality's backward hidden state therefore accumulates context from those that follow it in the reversed sequence. The averaged representation

$$H = \frac{h^{\rightarrow} + h^{\leftarrow}}{2}$$

remains order-dependent: both the forward and backward context seen by modality i change qualitatively with its position in the arbitrary concatenation, so the same physical modality receives a structurally different representation depending solely on where it was placed in m.

2.2 Differentiable Modality Ordering via Gumbel-Sinkhorn

We replace the fixed concatenation order with a *learned, input-dependent permutation* computed differentiably via the Gumbel-Sinkhorn operator [3]. The key intuition is that the optimal ordering is itself a function of the data: for a sample in which audio is uninformative, placing it first in the forward SSM pass deposits unreliable context into the hidden states of all subsequent modalities, whereas placing it last confines the contamination to the final position.

Uniform channel projection

Different InitEncoders naturally produce output representations of different widths: for instance, an audio encoder may output $C'_{\text{audio}} = 64$ channels while a video encoder outputs $C'_{\text{video}} = 256$ channels. These *heterogeneous dimensions* prevent treating the modality outputs as interchangeable objects of the same type, which is a prerequisite for learning a permutation over them. To resolve this, a modality-specific affine map $f_i : \mathbb{R}^{C'_i} \to \mathbb{R}^{C'}$, defined by $f_i(v) = W_i^{\text{proj}} v + b_i^{\text{proj}}$ with $W_i^{\text{proj}} \in \mathbb{R}^{C' \times C'_i}$ and $b_i^{\text{proj}} \in \mathbb{R}^{C'}$, is appended to each InitEncoder to linearly project its output into a shared embedding space of dimension C'. After this projection, $u_i \in \mathbb{R}^{L' \times C'}$ for all $i \in \{1, \ldots, M\}$. The matrix U is then the concatenation of these uniformly-dimensioned representations along the channel axis: $U = u_1 \circ u_2 \circ \cdots \circ u_M \in \mathbb{R}^{L' \times MC'}$, where each u_i contributes exactly C' columns. As with the encoders themselves, the projection is shared between the expert and novice streams of a given feature type.

Score matrix

A *meta-network* ϕ is a small auxiliary network whose sole purpose is to predict how modalities should be ordered, operating on top of the main feature representations without participating in the primary task of engagement prediction. Concretely, ϕ maps the set $\{u_i\}_{i=1}^{M}$ to an $M \times M$ assignment *score* matrix $Z \in \mathbb{R}^{M \times M}$, where the entry Z_{ij} quantifies the fitness of placing modality j at sequence position i in the inter-modal BiMamba: a large value of Z_{ij} means the

meta-network believes modality j should be processed at step i. Each modality representation is first summarised by temporal mean-pooling:

$$\bar{u}_i = \frac{1}{L'} \sum_{t=1}^{L'} u_i^{(t)} \in \mathbb{R}^{C'}.$$

Two asymmetric bias-free projections into a score-embedding space of dimension d_s then produce query and key vectors:

$$q_i = W_Q \, \bar{u}_i \in \mathbb{R}^{d_s}, \qquad k_j = W_K \, \bar{u}_j \in \mathbb{R}^{d_s},$$

where $W_Q, W_K \in \mathbb{R}^{d_s \times C'}$ are learnable parameters shared across modalities. The score is the dot product

$$Z_{ij} = q_i^\top k_j.$$

Remark: the conventional scaled variant $Z_{ij} = q_i^\top k_j / \sqrt{d_s}$ is *not* currently applied. Because Z is subsequently divided by the annealing temperature τ, an unscaled score of large magnitude has the same effect as a small τ, which drives the Sinkhorn output towards a hard permutation before the annealing schedule intends it to. Introducing the $1/\sqrt{d_s}$ factor is an open item.

The asymmetry of the query-key factorisation is intentional: it allows the model to distinguish between a modality acting as a *context supplier* (the key role) and a modality acting as a *context receiver* (the query role), and assigns independent scores to each direction. A symmetric formulation $Z_{ij} = z_i^\top z_j$ would force $Z = Z^\top$, which in turn constrains the learned permutation matrix P to be symmetric. The only symmetric permutation matrices are *involutions* - permutations that are their own inverse ($\pi = \pi^{-1}$), corresponding to products of disjoint transpositions (swaps). This would unnecessarily halve the space of learnable orderings, excluding, for example, cyclic permutations of three or more modalities.

Gumbel perturbation

The Gumbel distribution is classically used in extreme value theory [5] to model the asymptotic distribution of the maximum of a sequence of independent and identically distributed random variables. Its probability density function for location μ and scale β is given by:

$$f(x) = \frac{1}{\beta} e^{-(x-\mu)/\beta} e^{-e^{-(x-\mu)/\beta}}$$

In deep learning, the Gumbel-Max trick [4] leverages this distribution to draw discrete samples from a categorical distribution in a fully differentiable manner. By adding independent Gumbel noise to unnormalized log-probabilities (scores), the deterministic argmax of these perturbed values becomes mathematically equivalent to drawing a sample from the true underlying multinomial distribution.

To generate standard Gumbel noise ($\mu = 0, \beta = 1$) programmatically, we apply inverse transform sampling. This technique maps a standard uniform random variable to a target distribution by passing it through the inverse of that target's cumulative distribution function (CDF). Given that the standard Gumbel CDF is $F(x) = e^{-e^{-x}}$, setting a uniform random variable $U \sim \text{Uniform}(0, 1)$ equal to $F(x)$ and solving explicitly for x yields the inverse mapping $x = -\ln(-\ln U)$. In practice U is drawn from the open interval $(\epsilon, 1 - \epsilon)$ with $\epsilon = 10^{-6}$, since the double logarithm diverges at both endpoints. Thus, to enable stochastic exploration of orderings during training, the score matrix Z is perturbed with i.i.d. standard Gumbel noise and scaled by a temperature parameter $\tau > 0$:

$$\tilde{Z}_{ij} = \frac{Z_{ij} + G_{ij}}{\tau}, \qquad G_{ij} = -\ln(-\ln U_{ij}), \quad U_{ij} \overset{\text{i.i.d.}}{\sim} \text{Uniform}(0, 1).$$

The temperature τ is annealed over the course of training to progressively concentrate probability mass towards a single hard permutation. We adopt a standard exponential decay schedule, updating the temperature at each epoch t via $\tau_t = \max(\tau_\infty, \tau_0 \cdot \gamma^t)$, where $\gamma \in (0, 1)$ represents the decay rate, starting from an initial exploration temperature τ_0 down to a tight categorical approximation threshold τ_∞. Concrete values are listed in Section 5.

Sinkhorn normalisation

The **Birkhoff-von Neumann theorem** [6] establishes that the set of doubly-stochastic matrices (square matrices with non-negative real entries whose rows and columns each sum to exactly 1) constitutes a convex polytope whose extreme points are precisely the set of discrete permutation matrices. Geometrically, a convex polytope is a bounded, multi-dimensional solid defined by flat facets, and its extreme points represent its absolute vertices. Because any interior point of a convex polytope can be expressed uniquely as a convex combination (a weighted average) of its vertices, any doubly-stochastic matrix can be decomposed into a convex combination of hard permutation matrices. This structural property makes the Birkhoff polytope a mathematically principled continuous relaxation of discrete permutations.

A standard Softmax operator is insufficient in this context because it operates independently across a single dimension, ensuring only that the rows sum to 1 (yielding a row-stochastic matrix). In an inter-modal fusion framework where rows represent sequence positions and columns map to distinct modalities, a row-stochastic matrix introduces severe structural redundancies. It permits multiple sequence positions to concurrently assign high probability mass to the exact same dominant modality column (such as audio), which causes the network to attend to duplicate representations while completely suppressing and ignoring other available modalities.

Constraining both rows and columns to sum to 1 eliminates this issue by enforcing a strict continuous one-to-one assignment map. Following [3], the doubly-stochastic matrix P is obtained by the iterative Sinkhorn-Knopp algorithm, which alternates row and column normalisation of $\exp(\tilde{Z})$ until convergence.

Log-domain implementation. We do *not* evaluate $\exp(\tilde{Z})$ explicitly. With real feature magnitudes the unscaled scores Z_{ij} reach 10^4–10^5, so $\exp(\tilde{Z})$ overflows to $+\infty$ in single precision; the subsequent row normalisation then computes ∞/∞ and returns NaN, which propagates through the entire network and destroys training. We therefore carry out the alternating normalisation entirely in log-space.

The operation that replaces division is the *log-sum-exp*, defined for a finite set of reals $\{x_1, \ldots, x_n\}$ as

$$\operatorname*{logsumexp}_j x_j \;:=\; \log \sum_{j=1}^{n} \exp(x_j),$$

that is, the logarithm of the sum of the exponentials. Its role here follows from a single identity. If a set of quantities is held in logarithmic form, $x_j = \log p_j$, then normalising them to sum to one, $p_j \mapsto p_j / \sum_{j'} p_{j'}$, becomes in logarithms

$$\log\left(\frac{p_j}{\sum_{j'} p_{j'}}\right) = \log p_j - \log \sum_{j'} p_{j'} = x_j - \operatorname*{logsumexp}_{j'} x_{j'}.$$

A normalisation is therefore a *subtraction* in log-space, never a division, and the sums themselves are never formed. Crucially, log-sum-exp is evaluated by first factoring out the largest element, $m = \max_j x_j$,

$$\operatorname*{logsumexp}_j x_j = m + \log \sum_j \exp(x_j - m),$$

which is algebraically exact and guarantees every exponential argument is non-positive, so no term can exceed 1 and overflow cannot occur regardless of how large the x_j become.

Writing $\log Q^{(k)}$ for the matrix obtained after the row pass of iteration k, and $\log P^{(k+1)}$ for the result after the subsequent column pass, the scheme is

$$\log P^{(0)} = \tilde{Z},$$

$$\log Q_{ij}^{(k)} = \log P_{ij}^{(k)} - \operatorname*{logsumexp}_{j'} \log P_{ij'}^{(k)}, \qquad \log P_{ij}^{(k+1)} = \log Q_{ij}^{(k)} - \operatorname*{logsumexp}_{i'} \log Q_{i'j}^{(k)},$$

for $k = 0, \ldots, n_{\text{iter}} - 1$, with $P = \exp\big(\log P^{(n_{\text{iter}})}\big)$ returned only at the end. The first update normalises each row (the inner index runs over columns j'), the second normalises each column (the inner index runs over rows i'); $Q^{(k)}$ is simply the intermediate that is row-stochastic but not yet column-stochastic, and it is discarded once the column pass consumes it. Alternating the two passes is what drives the matrix towards satisfying both constraints at once.

This is algebraically identical to Sinkhorn-Knopp on $\exp(\tilde{Z})$ - each step divides by a row or column sum, which by the identity above is a subtraction of its logarithm - but the largest intermediate quantity is $\max_j \tilde{Z}_{ij}$ rather than $\exp(\max_j \tilde{Z}_{ij})$. The exponential map is applied once, at the end, to values that are by construction non-positive log-probabilities, so the returned entries lie in $(0, 1]$ and the gradient operator $\partial P / \partial Z$ remains well-conditioned throughout optimisation.

Soft reordering

With P computed as above, the permuted representation for position i is the P-weighted convex combination of all intra-modal outputs. Concretely, the i-th row of P, denoted $P_{i\cdot} = (P_{i1}, \ldots, P_{iM})$, defines a probability distribution over the M modalities; since P is doubly-stochastic, we have $P_{ij} \geq 0$ and $\sum_j P_{ij} = 1$ for every i. The soft assignment is then:

$$\tilde{u}_i = \sum_{j=1}^{M} P_{ij}\, u_j \in \mathbb{R}^{L' \times C'}, \qquad i \in \{1, \ldots, M\}.$$

The entry $P_{ij} \in [0, 1]$ is the soft probability that modality j is assigned to position i in the inter-modal sequence; when P is close to a hard permutation matrix (as happens after temperature annealing), exactly one P_{ij} per row will be close to 1 and the rest close to 0, recovering a near-discrete assignment. The stacked representation is accordingly

$$\tilde{\mathcal{U}} \in \mathbb{R}^{M \times L' \times C'}, \qquad \tilde{\mathcal{U}}_i = \tilde{u}_i,$$

and the inter-modal BiMamba operates on $\tilde{\mathcal{U}}$ in place of the unpermuted stack; all equations in Section 1.3.2 remain unchanged under this substitution. Note that P is computed per sample: the ordering is input-dependent, so different sessions may resolve to different modality orderings.

Inference

At test time, Gumbel noise is suppressed ($G_{ij} = 0$), so the permutation becomes a deterministic function of the input; the soft doubly-stochastic P is used directly for the reordering.

The following hard-assignment variant is described for completeness but is not part of the current implementation. If a hard permutation is required (e.g. for latency-critical deployment), the optimal discrete assignment is recovered by solving the linear assignment problem over the score matrix Z:

$$\pi^* = \arg\min_{\pi \in S_M} \sum_{i=1}^{M} \big(-Z_{i,\pi(i)}\big),$$

where S_M denotes the symmetric group - the set of all $M!$ bijections from $\{1, \ldots, M\}$ to itself, i.e. all possible orderings of the M modalities. This minimisation is solved by applying the Hungarian algorithm to $-Z$ in $O(M^3)$ time, which is negligible for the small modality counts typical in multimodal sensing.

Equivariance guarantee

Let $\rho \in S_M$ be applied to the inputs before the meta-network. Because Z_{ij} is computed from the data via the shared query-key projections, relabelling the inputs by ρ relabels the rows and columns of Z by ρ as well, and consequently the recovered hard permutation transforms as $\pi^*_\rho = \rho \circ \pi^*$. The permuted sequence fed to the inter-modal BiMamba is therefore the same physical ordering of modalities regardless of how the practitioner originally numbered them. Consequently, the fused representation $H = \text{BiMamba}_{\text{modality}}(\tilde{\mathcal{U}})$ is invariant to the practitioner's arbitrary choice of modality ordering, provided the model has converged to a stable optimal ordering π^*. This guarantee is stated for the hard-assignment variant; under the soft doubly-stochastic P actually used, the corresponding statement is approximate and becomes exact only in the low-temperature limit $\tau \to 0$, where P converges to a permutation matrix.

3 Evaluation

3.1 Concordance Correlation Coefficient (CCC)

The primary evaluation metric is the Concordance Correlation Coefficient [12], which measures agreement between predicted and ground-truth engagement trajectories:

$$\text{CCC}(\hat{y}, y) = \frac{2\rho\, \sigma_{\hat{y}}\, \sigma_y}{\sigma_{\hat{y}}^2 + \sigma_y^2 + (\mu_{\hat{y}} - \mu_y)^2},$$

where ρ is the Pearson correlation between predictions $\hat{y}$ and targets y, $\sigma_{\hat{y}}, \sigma_y$ are their standard deviations, and $\mu_{\hat{y}}, \mu_y$ are their means. $\text{CCC} \in [-1, 1]$, with 1 indicating perfect concordance. Unlike Pearson correlation alone, CCC penalises both scale and location shifts.

Following the challenge protocol [13], the headline score is the **unweighted mean CCC across the four held-out test sets** - NoXi, NoXi+J, NoXi (Additional Languages), and MPIIGroupInteraction:

$$\text{CCC}_{\text{combined}} = \frac{1}{4} \sum_{d \in \mathcal{D}} \text{CCC}_d, \qquad \mathcal{D} = \{\text{NoXi},\ \text{NoXi+J},\ \text{NoXi-Add},\ \text{MPIIGI}\}.$$

Because the mean is unweighted, a domain on which the model transfers poorly is not offset by strong in-domain performance elsewhere; cross-domain robustness is therefore weighted as heavily as in-domain accuracy.

The correlation ceiling

CCC can be decomposed into two distinct terms: one that measures how well the prediction *orders* the target, and another that measures how well its scale and location *match* the target. Define the scale ratio and the normalised mean offset

$$\varphi := \frac{\sigma_{\hat{y}}}{\sigma_y} > 0, \qquad \omega := \frac{\mu_{\hat{y}} - \mu_y}{\sqrt{\sigma_{\hat{y}} \sigma_y}},$$

and divide numerator and denominator of the CCC by $\sigma_{\hat{y}} \sigma_y$:

$$\frac{2\rho}{\dfrac{\sigma_{\hat{y}}}{\sigma_y} + \dfrac{\sigma_y}{\sigma_{\hat{y}}} + \dfrac{(\mu_{\hat{y}} - \mu_y)^2}{\sigma_{\hat{y}} \sigma_y}} = \rho \cdot C_b, \qquad C_b := \frac{2}{\varphi + \varphi^{-1} + \omega^2}.$$

Here ρ is the Pearson correlation, which we term the *precision* of the predictor, and C_b the bias-correction factor, its *accuracy*.

Proposition. $\mathrm{CCC}(\hat{y}, y) \leq \rho$, with equality iff $\sigma_{\hat{y}} = \sigma_y$ and $\mu_{\hat{y}} = \mu_y$.

Proof. For any $\varphi > 0$ the arithmetic-geometric mean inequality gives $\varphi + \varphi^{-1} \geq 2\sqrt{\varphi \cdot \varphi^{-1}} = 2$, with equality if and only if $\varphi = \varphi^{-1}$ ($\varphi = 1$). Since also $\omega^2 \geq 0$ with equality if f $\mu_{\hat{y}} = \mu_y$, the denominator of C_b satisfies $\varphi + \varphi^{-1} + \omega^2 \geq 2$, hence $C_b \leq 1$ and $\mathrm{CCC} = \rho\, C_b \leq \rho$. Both equality conditions hold simultaneously precisely when $\sigma_{\hat{y}} = \sigma_y$ and $\mu_{\hat{y}} = \mu_y$. □

Implication. The correlation ρ is a ceiling on the CCC that no post-hoc transformation of the predictions can raise. The accuracy factor C_b can be driven towards its maximum of 1 by matching the prediction's standard deviation and mean to those of the target, an essentially mechanical calibration; but ρ is invariant under any affine rescaling of $\hat{y}$ and depends only on how the predictions are *ordered* relative to the targets. Once a model is approximately calibrated, therefore, the entire remaining margin lies in improving that ordering - a property of the model's discriminative capacity, not of its output scale. This is what recommends architectural and optimisation choices that lengthen the temporal horizon over which evidence is integrated (Section 1.3.1) and that make cross-modal context genuinely available (Section 1.3.2), and it motivates supplementing the pointwise likelihood with a discriminative training term.

Implication. The Pearson correlation ρ sets an upper limit on the CCC that cannot be increased by tweaking or post-processing the predictions after the model has run. The accuracy factor C_b can easily reach its maximum value of 1 simply by adjusting the predictions' mean and standard deviation to match the real targets. However, scaling or shifting the predictions ($\hat{y}$) mathematically does not change ρ at all, because ρ depends purely on whether the predictions are placed in the correct relative order.

Once a model is properly calibrated, any further boost in performance must come from improving that target ordering. This relies directly on the model's ability to tell different states apart, rather than the scale of its outputs. This insight justifies structural and optimization decisions that expand the model's memory over longer time spans and ensure that information moves cleanly across different modalities. It also provides a strong reason to add a discriminative loss term alongside standard point-by-point likelihood training.

3.2 Conditional Demographic Disparity (CDD)

To quantify fairness across demographic groups we adopt the Conditional Demographic Disparity [14, 13], which asks whether, *at the same ground-truth engagement level*, the model systematically over- or under-predicts for one group relative to another. Conditioning on y is what distinguishes CDD from a raw demographic-parity gap: it separates genuine model bias from legitimate differences in the underlying engagement distributions of the groups.

For gender,

$$\mathrm{CDD}_G = \mathbb{E}\big[\hat{y} \,\big|\, \text{male},\, y\big] - \mathbb{E}\big[\hat{y} \,\big|\, \text{female},\, y\big],$$

where positive values indicate higher predictions for males than for females at the same y, and negative values the reverse. For language, each language L is compared against the pooled expectation across all languages at the same y:

$$\mathrm{CDD}_L = \mathbb{E}\big[\hat{y} \,\big|\, L,\, y\big] - \mathbb{E}\big[\hat{y} \,\big|\, y\big].$$

Both quantities are **signed** and centred at 0; values closer to zero indicate smaller disparity, and the sign indicates the direction of the bias. This is a substantive difference from an absolute-value formulation: the sign carries the information about *which* group is favoured, and averaging absolute values across groups would discard it.

Estimation. Since y is continuous, the conditional expectations are approximated by binning the ground truth into n_b equal-frequency (quantile) bins and averaging the within-bin group differences, weighted by bin prevalence:

$$\widehat{\mathrm{CDD}} = \frac{\sum_{b=1}^{n_b} |\mathcal{B}_b| \cdot \left(\bar{\hat{y}}_b^A - \bar{\hat{y}}_b^B\right)}{\sum_{b=1}^{n_b} |\mathcal{B}_b|},$$

where $\mathcal{B}_b$ is the set of frames whose target falls in bin b, and $\bar{\hat{y}}_b^A, \bar{\hat{y}}_b^B$ are the mean predictions of the two groups within that bin. Bins in which either group is unrepresented are skipped, since no comparison is defined there. We use $n_b = 10$.

Applicability. CDD_G requires exactly two gender codes to be present in the evaluated split, and CDD_L requires at least two languages; where a split does not satisfy these conditions the corresponding metric is undefined and is reported as such rather than as zero. Note also that the sign of CDD_G depends on which annotation code denotes male, which must be resolved against the corpus metadata; the magnitude is invariant to this choice. We acknowledge, following [13], that gender is not binary; the corpora provide only male/female self-identifications.

Because every step in the estimator is differentiable, CDD is not only measured but trained against: a squared form of it appears as the fairness term of Section 2.5.

4 Hyperparameters

The following values define the reference configuration. All are exposed as command-line overrides.

Symbol	Description	Value
Data		
L	Window length (frames; 10 s at 25 Hz)	250
–	Active feature types $\lvert\mathcal{F}\rvert$ (core subset)	4
M	Role-feature pairs $= 2\lvert\mathcal{F}\rvert$	8
ε	Target clipping bound	10^{-6}
Architecture		
C'	Shared channel width after projection	128
N	SSM state dimension	16
D_C	Depthwise convolution kernel size	4
d_h	Beta head hidden width	128
–	Total parameters (core subset)	≈ 7.2M
$\Delta_{\min}$	Minimum timestep at initialisation	10^{-3}
$\Delta_{\max}$	Maximum timestep at initialisation	10^{-1}
–	A initialisation scheme	S4D-Real
Gumbel-Sinkhorn		
d_s	Query/key projection dimension	64
n_{iter}	Sinkhorn normalisation iterations	20
τ_0	Initial temperature	1.0
τ_∞	Minimum temperature	0.1
γ	Temperature decay rate per epoch	0.95
Optimisation		
η	Learning rate (Adam)	10^{-3}
λ_u	Unimodal deep-supervision weight	0.5
–	Batch size	8
–	Maximum epochs	50
–	Early-stopping patience (epochs, on val CCC)	10
λ_c	Weight of the ranking (CCC) term	1.0
λ_u	Unimodal deep-supervision weight (before $1/M$)	0.5
λ_f	Fairness penalty weight	0.05
β_w	Likelihood reweighting exponent	0.5
Test-time adaptation		
λ	Pseudo-label supervision weight in $\mathcal{L}_{TTA}$	1.0
p	Percentile rank for adaptive thresholds	0.8
Evaluation		
n_b	Ground-truth bins for CDD estimation	10

The learning rate $\eta = 10^{-3}$ is the current default but is under review: a full run at this rate reached its best validation CCC at epoch 2 and then flattened for ten consecutive epochs until early stopping, while the training loss continued to fall - the signature of steps that are too large once input standardisation permits gradients to propagate properly. Lower values (5×10^{-4} and below) are being evaluated.

References

[1] Ziyu Jia et al., *A Multimodal BiMamba Network with Test-Time Adaptation for Emotion Recognition Based on Physiological Signals*, 39th Conference on Neural Information Processing Systems (NeurIPS 2025).

[2] Albert Gu and Tri Dao, *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, arXiv preprint arXiv:2312.00752 (2024).

[3] Gonzalo Mena et al., *Learning Latent Permutations with Gumbel-Sinkhorn Networks*, International Conference on Learning Representations (ICLR 2018).

[4] Eric Jang, Shixiang Gu, and Ben Poole, *Categorical Reparameterization with Gumbel-Softmax*, International Conference on Learning Representations (ICLR 2017).

[5] Emil Julius Gumbel, *Statistical Theory of Extreme Values and Some Practical Applications*, National Bureau of Standards Applied Mathematics Series, 33 (1954).

[6] Garrett Birkhoff, *Tres observaciones sobre el algebra lineal*, Universidad Nacional de Tucumán, Revista, Serie A, 5:147–151 (1946).

[7] Aitchison, J., & Shen, S. M. (1980). *Logistic-normal distributions: Some properties and uses.* Biometrika, 67(2), 261–272.

[8] P. Kumaraswamy, *A generalized probability density function for double-bounded random processes*, Journal of Hydrology, 46, 79–88 (1980).

[9] Guilherme Pumi, Cleber Rauber, and Fabio M. Bayer, *Kumaraswamy regression model with Aranda-Ordaz link function*, TEST: An Official Journal of the Spanish Society of Statistics and Operations Research, Springer;Sociedad de Estadística e Investigación Operativa, 29(4), 1051–1071, December (2020).

[10] Yaniv Romano, Evan Patterson, and Emmanuel J. Candès, *Conformalized Quantile Regression*, Advances in Neural Information Processing Systems (NeurIPS 2019), 32 (2019).

[11] Isaac Gibbs and Emmanuel Candès, *Adaptive Conformal Inference Under Distribution Shift*, Advances in Neural Information Processing Systems (NeurIPS 2021), 34, 1660–1672 (2021).

[12] Lawrence I-Kuei Lin, *A Concordance Correlation Coefficient to Evaluate Reproducibility*, Biometrics, 45(1), 255–268 (1989).

[13] Daksitha Withanage Don, Marius Funk, Michal Balazia, Huajian Qiu, Shogo Okada, François Brémond, Jan Alexandersson, Andreas Bulling, Elisabeth André, and Philipp Müller, *MultiMediate '25: Cross-cultural Multi-domain Engagement Estimation*, Proceedings of the 33rd ACM International Conference on Multimedia (MM '25), 14150–14155 (2025). doi:10.1145/3746027.3762076.

[14] Sandra Wachter, Brent Mittelstadt, and Chris Russell, *Why fairness cannot be automated: Bridging the gap between EU non-discrimination law and AI*, Computer Law and Security Review, 41, 105567 (2021). doi:10.1016/j.clsr.2021.105567.

[15] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton, *Layer Normalization*, arXiv preprint arXiv:1607.06450 (2016).

[16] Sergey Ioffe and Christian Szegedy, *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*, International Conference on Machine Learning (ICML 2015), 448–456.

[17] Angelo Cafaro, Johannes Wagner, Tobias Baur, Soumia Dermouche, Mercedes Torres Torres, Catherine Pelachaud, Elisabeth André, and Michel F. Valstar, *The NoXi Database: Multimodal Recordings of Mediated Novice-Expert Interactions*, Proceedings of the 19th ACM International Conference on Multimodal Interaction, 350–359 (2017). doi:10.1145/3136755.3136780.

[18] Marius Funk, Shogo Okada, and Elisabeth André, *Multilingual Dyadic Interaction Corpus NoXi+J: Toward Understanding Asian-European Non-verbal Cultural Characteristics and their Influences on Engagement*, Proceedings of the ACM International Conference on Multimodal Interaction, 224–233 (2024). doi:10.1145/3678957.3685757.

[19] Albert Gu, Ankit Gupta, Karan Goel, and Christopher Ré, *On the Parameterization and Initialization of Diagonal State Space Models*, Advances in Neural Information Processing Systems (NeurIPS 2022), 35, 35971–35983.

[20] Alan V. Oppenheim, Alan S. Willsky, and S. Hamid Nawab, *Signals and Systems*, 2nd edition, Prentice Hall (1997).

[21] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius, *On the Pitfalls of Heteroscedastic Uncertainty Estimation with Probabilistic Neural Networks*, International Conference on Learning Representations (ICLR 2022).